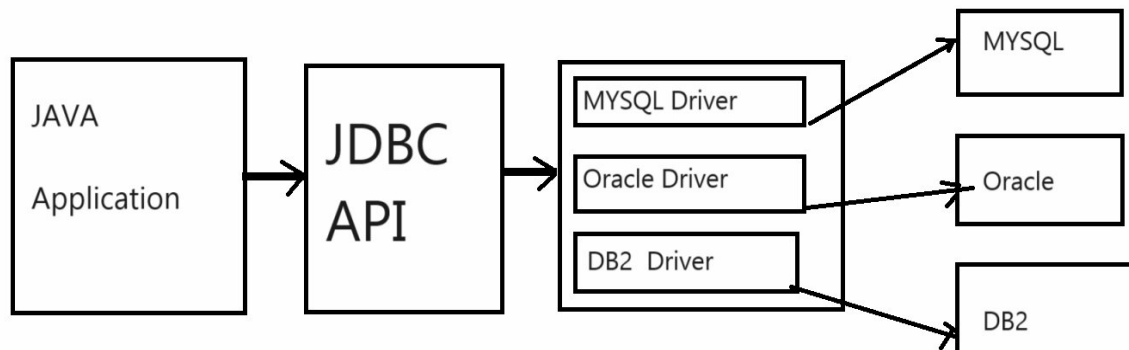


JDBC

JDBC stands java database connectivity it is API or some inbuilt packages, classes and interfaces which help us to connect our java application with database

Architecture of JDBC Program



How to works with JDBC or Steps to work with JDBC

1. Install any Database application or software like MYSQL, Oracle etc

2. Add java.sql package in application

```
import java.sql.*;
```

Because java.sql package is package which contain all classes and interfaces required for JDBC

3. Register the driver with application

Q. what is driver?

driver are the client side adapters which is used for convert your request to protocol for database communication called as driver means we can driver contain some logic or protocol which help us to connect your java application with your database application.

Types of Driver

Type1: it is JDBC-ODBC bridge driver

Type2: it is native interface driver or Native API driver (partially java driver)

Type3: it is network Protocol driver (fully java driver)

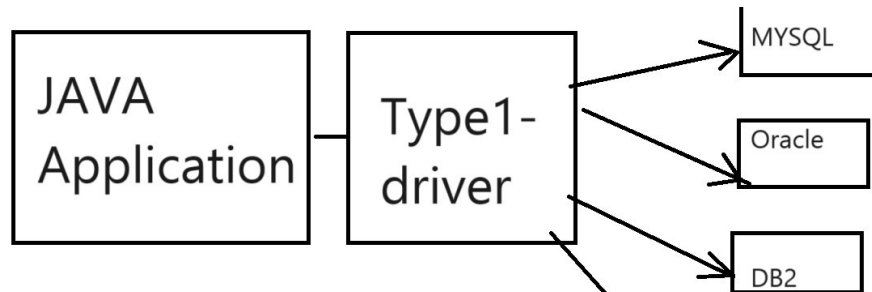
Type4: it is thin driver (fully java driver)

Note: we will discuss architecture of every driver at the end of topic

Note: We get driver in the form classes means we can say driver are the some inbuilt classes which contain database communication logics means in short we can say if we want to use driver means we use class internally and if we want to use any class we need to create its object means if we want to driver we need to create driver class object.

Now we want to discuss about type-1 driver

Type1 driver is ODBC so ODBC stands for open database connectivity it is concept launch by Microsoft using type1 driver we can connect your java application with any database means we can say type1 driver is database independent driver.



If we want to use this driver practically we have one inbuilt class name as `sun.jdbc.odbc.JdbcOdbcDriver`
Here `sun.jdbc.odbc` is package and `JdbcOdbcDriver` is classname

```
import sun.jdbc.odbc.*;
JdbcOdbcDriver d = new JdbcOdbcDriver();
```

Note: Type-1 Driver removed from java version JDK 1.8

Reason or Limitation of Type1 Driver

- a) Type 1 driver is platform dependent means it works with only windows
- b) Type 1 driver design in C and C++
- c) Type 1 driver not support to network programming means we cannot connect your application with remote database using type-1 driver

Benefits of Type-1 Driver

- a) It is present in JDK not need to installed manually or download manually form some other place.
- b) It is database independent means using type-1 you can connect your java application with any other database.

Now Here we will use type-4 driver for connect java application with database which is known as thin driver.

This type of driver classes provided by database vendor in the form .jar file means type4 driver class is different for every database.

Example: Suppose Consider we want to connect java application with MYSQL Database using type-4 driver then we need to download the MYSQL Connector.jar file

Suppose consider we have MYSQL 8 then we need to download the .jar mysql-connector-8 version

Q. what is .jar file?

.jar stands for JAVA Archive files basically it are collection of packages, classes and interfaces and it contain only .class files.

Q. What is benefit of .jar file?

.jar file help us to create the Reusable API or Reusable classes and interfaces means suppose consider we have some logic or some API and if we want to reuse that API by some other then we can create .jar file and hand over to other for reuse API.

So Now we want to download the mysql connector.jar file for connect our java program with mysql database tool.

You can download mysql connector.jar file using this link

jar-download.com/download-handling.php

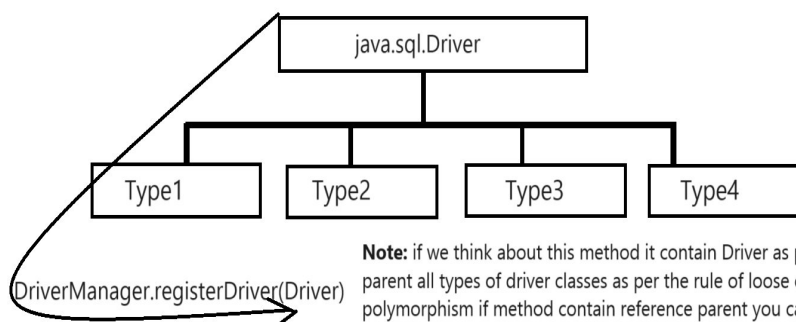
If we think about mysql -connector .jar file it contain class which work as driver class name as com.mysql.cj.jdbc.Driver

Here com.mysql.cj.jdbc is package and Driver is class means if we want to use this driver you have to create its object

```
com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
```

Once we create object of Driver class you can register driver with Java application and if we want to register Driver with java application we have one inbuilt class name as DriverManager and its contain one static method name as registerDriver() which help us to register driver with java application.

Syntax: DriverManager.registerDriver(Driver): if we think about this method i.e registerDriver() it contain one parameter name as Driver which is interface from java.sql package and it is implemented all types of driver classes



Note: if we think about this method it contain Driver as parameter and Driver is parent all types of driver classes as per the rule of loose coupling and dynamic polymorphism if method contain reference parent you can pass any child reference in it means registerDriver() can accept the reference of as parameter type-1,type-2,type-3 type -4 means it is common method for register any type of driver if in future java launch new driver class and mark its parent as Driver then registerDriver() can work with new driver class also.

Steps to use the .jar file and register the driver by using eclipse

- Create Java Project using eclipse.
- Download the .jar file and store on some location in your system
- Import .jar file in project using eclipse or add .jar file in project using eclipse.

steps to import .jar files

right click on JRE system library ---- select built-path option - configure build path ---- select class path (not module path) --- click on add external .jar file ---- select .jar file from system and click on apply button

After import .jar we get following type of folder structure



Once we import .jar file you can write code and register driver with java application

package org.techhub;

import java.sql.*;

public class RegisterDriverApp {

public static void main(String[] args) **throws** SQLException{

 com.mysql.cj.jdbc.Driver d= **new** com.mysql.cj.jdbc.Driver();

 DriverManager.registerDriver(d);

 System.out.println("Driver Registered Successfully.....");

 }

}

4. Establish Connection between java application and database application

If we want to connect your java application with database application we have interface from java.sql package name as Connection and if we want to create reference of Connection interface for that we have method name as DriverManager.getConnection()

getConnection() is a static method of DriverManager and this method return reference of Connection interface.

Syntax: `Connection ref = DriverManager.getConnection(String url,String username,String password);`

Parameter details

String url: url means path where we store our database or location where our database is present.

String username: it is our database username

String password: it our database password.

Note: if we use think about MYSQL we have url like as

`jdbc:mysql://localhost:portnumber/databasename`

Example: `jdbc:mysql://localhost:3306/mysql`

`jdbc:mysql` is protocol

`localhost` or `ipaddress`: if your database present on same machine where your project exist then use `localhost` or use `id 127.0.0.1` and if your database present on remote machine or on different machine then you can specify ip address other than `localhost`

`3306` it is port number of MYSQL database and it is default and if user change the port number at the time of installation you can specify your port except `3306`

`mysql` is your database and user can give his own database name also.

```
package org.techhub;
import java.sql.*;
public class RegisterDriverApp {
    public static void main(String[] args) throws SQLException{
        com.mysql.cj.jdbc.Driver d= new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/mysql","root","root");
        if(conn!=null) {
            System.out.println("Database is connected");
        }
        else {
            System.out.println("Some problem is there.....");
        }
    }
}
```

5. Work with Database

If we want to work with database we have some following interfaces given below and classes

Interfaces for work with Database

- a. Statement
- b. PreparedStatement
- c. ResultSet
- d. CallableStatement
- e. ResultSetMetaData
- f. DatabaseMetaData
- g. Connection
- f. ParameterMetaData

etc

Classes

- 1. DriverManager
- 2. Clob
- 3. Blob

Now we want to discuss about the Statement interface

Statement is interface from java.sql package which is used for perform all DDL and DML operation on database like as create, alter, insert, delete, upate etc

Steps to work with Statement interface

a) Add java.sql package in application

import java.sql.*;

b) Create Reference of Statement interface

If we want to create reference of Statement interface we have method name as `createStatement()` and it is member of Connection interface and this method return reference of Statement interface

Syntax: Statement ref = connref.createStatement();

Sample code

```
package org.techhub;
import java.sql.*;
public class InsertApplication {
    public static void main(String[] args) throws SQLException{
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/mysql",
"root", "root");
        if(conn!=null) {
            System.out.println("database is connected");
            Statement stmt=conn.createStatement();
```

```

    }
    else {
        System.out.println("Some problem is there.....");
    }
}
}

```

c) Use its method to execute database statement or queries

int executeUpdate(String sqlStatement): this method can execute the all DDL and DML statement except select like as using executeUpdate you can perform insert,delete, update etc operation. This method return 1 when operation executed successfully otherwise return 0.

ResultSet executeQuery(String selectStatement): this method can execute the select statement and return reference of ResultSet interface.

Example: We want to create table name as Employee with field id, name and salary and insert record in database table using Java program.

Steps.

a) Open MYSQL

b) Create Database Example: create database jdbcmayjunejuly

c) Use or Select Database: Example: use jdbcmayjunejuly

d) Create table in database Example: create table employee(id int(5),name varchar(200),salary int(5))

f) Write JDBC Program and insert record in database using Statement interface

```

1 package org.techhub;
2 import java.sql.*;
3 public class InsertApplication {
4     public static void main(String[] args) throws SQLException{
5         com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
6         DriverManager.registerDriver(d);
7         Connection conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly", "root", "root");
8         if(conn!=null) {
9             System.out.println("database is connected");
10            Statement stmt=conn.createStatement();
11            int value=stmt.executeUpdate("insert into employee values(1,'ram',10000)");
12            if(value>0) {
13                System.out.println("Record Save Success.....");
14            }
15            else {
16                System.out.println("Some problem is there.....");
17            }
18        }
19        else {
20            System.out.println("Some problem is there.....");
21        }
22    }
23 }

```

save

eid	name	salary
1	ram	10000

1 row in set (0.03 sec)

If we think about above code we insert fix data in database table we want to accept the input from keyboard and after that we want to save in database

```
1 package org.techhub;
2 import java.sql.*;
3 import java.util.*;
4 public class InsertApplication {
5     public static void main(String[] args) throws SQLException{
6         com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
7         DriverManager.registerDriver(d);
8         Connection conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly", "root", "root");
9         if(conn!=null) {
10             System.out.println("database is connected");
11             Statement stmt=conn.createStatement();
12             Scanner xyz = new Scanner(System.in);
13             System.out.println("Enter name id and salary of employee");
14             String name=xyz.nextLine();
15             int id=xyz.nextInt();
16             int sal=xyz.nextInt();
17             int value=stmt.executeUpdate("insert into employee values(id,'name',sal)");
18             if(value>0) {
19                 System.out.println("Record Save Success.....");
20             }
21             else {
22                 System.out.println("Some problem is there.....");
23             }
24         }
25         else {
26             System.out.println("Some problem is there.....");
27         }
28     }
29 }
30 }
```

```
Mon Feb 19 11:04:57 IST 2024 WARN: Establishing SSL connection without server's identity verification is not recomme
database is connected
Enter name id and salary of employee
ramesh
1
10000
Exception in thread "main" java.sql.SQLException: Unknown column 'id' in 'field list'
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:118)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:95)
    at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)
    at com.mysql.cj.jdbc.StatementImpl.executeUpdateInternal(StatementImpl.java:1393)
    at com.mysql.cj.jdbc.StatementImpl.executeUpdate(StatementImpl.java:2353)
```

if we think about above code we get exception at run time unknown column id in field list because we pass id as value and id is variable in java application so you cannot pass variable as value in query so when we want to pass run time parameter to SQL statement using JDBC we need to SQL Injection Technique means we need to separate variable from SQL statement using "+" symbol.

Source Code.

```
package org.techhub;
import java.sql.*;
import java.util.*;
public class InsertApplication {
    public static void main(String[] args) throws SQLException{
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly", "root", "root");
        if(conn!=null) {
            System.out.println("database is connected");
```



```

Statement stmt=conn.createStatement();
Scanner xyz = new Scanner(System.in);
System.out.println("Enter name id and salary of employee");
String name=xyz.nextLine();
int id=xyz.nextInt();
int sal=xyz.nextInt();
int value=stmt.executeUpdate("insert into employee values("+id+", '"+name+"', "+sal+"");
    if(value>0) {
        System.out.println("Record Save Success.....");
    }
    else {
        System.out.println("Some problem is there.....");
    }
}
else {
    System.out.println("Some problem is there.....");
}
}
}

```

```
mysql> select *from employee;
```

eid	name	salary
1	ram	10000
2	rajesh	20000

```

2 rows in set (0.00 sec)

```

Example: WAP to delete the employee record by using its id.

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class DeleteEmployeeById {
    public static void main(String[] args)throws SQLException {
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Statement stmt=conn.createStatement();
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter employee id");
            int empid=xyz.nextInt();

```

```

int value=stmt.executeUpdate("delete from employee where eid="+empid);
if(value>0) {
    System.out.println("Record Deleted Success.....");
}
else {
    System.out.println("Some problem is there.....");
}
}
else {
    System.out.println("Database is not connected");
}
}
}

```

mysql> select *from employee;

eid	name	salary
1	ram	10000
2	rajesh	20000

2 rows in set (0.00 sec)

mysql> select *from employee;

eid	name	salary
2	rajesh	20000

1 row in set (0.00 sec)

before delete

after delete

Example: WAP to update the employee name and its salary by using its id.

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class UpdateEmployeeById {
    public static void main(String x[]) throws Exception{
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly", "root", "root");
        if(conn!=null) {
            Statement stmt=conn.createStatement();
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter name id and salary of employee");
            String name=xyz.nextLine();
            int id=xyz.nextInt();

```

```

        int sal=xyz.nextInt();
        int value=stmt.executeUpdate("update employee set
name='"+name+"',salary='"+sal+"' where eid='"+id);
        if(value>0) {
            System.out.println("employee updated successfully.....");
        }
        else {
            System.out.println("Some problem in employee updation");
        }
    }
    else {
        System.out.println("Database is not connected");
    }
}
}

```

mysql> select *from employee;

eid	name	salary
2	rajesh	20000

1 row in set (0.00 sec)

mysql> select *from employee;

eid	name	salary
2	raju	20000

1 row in set (0.00 sec)

before update

After update

How to select data from database table

If we want to select data from database table we have executeQuery() method of Statement interface and this method return reference of ResultSet interface.

Q. what is ResultSet interface?

ResultSet is an interface from java.sql package which is used for hold data fetch by select query and provide access in java application.

ResultSet maintains cursor/pointer which points to a single row of the query results. Using navigational and getter methods provided by ResultSet, we can iterate and access database records one by one. ResultSet can also be used to update data.

ResultSet is the java.sql.ResultSet interface means the result set of a SQL query. It means a cursor is pointing a row of a table; it points to before the first row.

Methods of ResultSet interface

boolean next(): this method can check data present in database table or not if present return true otherwise return false.

String getString(String columnName): this method can fetch varchar type data from database table using its column name or using column index

or

String getString(int index): this method can fetch varchar data from table using column index.

If we have wild card selection using * in select command then first column consider as first index up to n.

Example: Suppose consider we have Employee table with column

id name salary

and we have select query like as

```
select *from employee;
```

Here id has 1 index , name has 2 and salary has 3rd index

Suppose consider if we execute the partial select command then from left to right give index to column means first left column has 1 index up to n

Example: select salary ,name from employee

here salary has 1 index even it is 3rd column in table and name has 2nd index means in partial selection we provide column index from to left it is not dependent on table structure.

int getInt(String columnName): this method can fetch integer type of data using column name

or

int getInt(int index): this method can fetch data of type integer using its column index.

float getFloat(String columnName): this method can fetch data of type float using its column name

or

float getFloat(int index): this method can fetch float type of data using its index.

etc

What are types of ResultSet?

There are three types of ResultSet available. If we do not declare any ResultSet that means we are calling TYPE_FORWARD_ONLY

1 Forward Only (ResultSet.TYPE_FORWARD_ONLY): cursor can move only forward.

This type of ResultSet instance can move only in the forward direction from the first row to the last row.

ResultSet can be moved forward one row by calling the next() method. We can obtain this type of ResultSet while creating Instance of Statement, PreparedStatement or CallableStatement.

Example:

```
Statement stmt =
connref.createStatement(ResultSet.TYPE_FORWARD_ONLY,ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("select * from tbluser");
```

2. Scroll insensitive (ResultSet.TYPE_SCROLL_INSENSITIVE): cursor can move forward and backward but not sensitive.

Scroll Insensitive ResultSet can scroll in both forward and backward directions. It can also be scrolled to an absolute position by calling the absolute() method. But it is not sensitive to data changes. It will only have data when the query was executed and ResultSet was obtained. It will not reflect the changes made to data after it was obtained.

```
Example: Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
        ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("select * from tbluser");
```

3. Scroll Sensitive (ResultSet.TYPE_SCROLL_SENSITIVE): cursor can move forward and backward, but it is sensitive.

Scroll Sensitive ResultSet can scroll in both forward and backward directions. It can also be scrolled to an absolute position by calling the absolute() method. But it is sensitive to data changes. It will reflect the changes made to data while it is open.

```
Example: Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
        ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("select * from tbluser");
```

Example: we want to fetch all employee record from employee table using JDBC

```
package org.techhub;
import java.sql.*;
public class SelectQueryApplication {
    public static void main(String[] args)throws SQLException {
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Statement stmt=conn.createStatement();
            ResultSet rs=stmt.executeQuery("select *from employee");
            while( rs.next() )
            {
                int empId=rs.getInt("eid");

                String empName=rs.getString("name");

                int empSalary=rs.getInt("salary");
```

```

        System.out.println(empId+"\t"+empName+"\t"+empSalary);
    }
}
else {
    System.out.println("Database is not connected");
}
}
}

```

```

Mon Feb 15 11:31:24 IST 2016
2      raju      20000
1      ABC       10000
3      STV       20000
4      XYZ       40000

```

Program output

```

mysql> select *from employee;
+----+-----+-----+
| eid | name  | salary |
+----+-----+-----+
| 2   | raju  | 20000  |
| 1   | ABC   | 10000  |
| 3   | STV   | 20000  |
| 4   | XYZ   | 40000  |
+----+-----+-----+
4 rows in set (0.00 sec)

```

Actual Database Table.

Example: WAP to fetch employee data using its id.

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class JdbcWithWhereClauseApp {
    public static void main(String[] args) throws SQLException{
        com.mysql.cj.jdbc.Driver d= new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Statement stmt=conn.createStatement();
            Scanner xyz = new Scanner(System.in);
            System.out.println("enter employee id");
            int id=xyz.nextInt();
            ResultSet rs=stmt.executeQuery("select *from employee where eid="+id);
            if(rs.next()) {
                System.out.println(rs.getInt(1)+"\t"+rs.getString(2)+"\t"+rs.getInt(3));
            }
            else {
                System.out.println("Employee not found");
            }
        }
    }
}
else {

```

```

        System.out.println("database is not connected");
    }
}
}

```

Example: WAP to create table name as register with column id,name , email, contact ,username and password

```

mysql> create table register(rid int(5) primary key auto_increment,name varchar(200) not null,email
varchar(200) not null unique,contact varchar(200) not null unique,username varchar(200) not null
unique,password varchar(200) not null unique);
Query OK, 0 rows affected, 1 warning (0.20 sec)

```

```

package org.techhub;

```

```

import java.util.*;
import java.sql.*;

```

```

public class RegisterApplication {
    public static void main(String[] args) throws SQLException {
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly", "root", "root");
        if (conn != null) {
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter name ");
            String name = xyz.nextLine();
            System.out.println("Enter Email");
            String email = xyz.nextLine();
            System.out.println("Enter contact");
            String contact = xyz.nextLine();
            System.out.println("Enter username");
            String username = xyz.nextLine();
            System.out.println("Enter password");
            String password = xyz.nextLine();
            Statement stmt = conn.createStatement();
            int value = stmt.executeUpdate("insert into register values('0','" + name + "','" +
email + "','" + contact
                                + "','" + username + "','" + password + "')");
            if (value > 0) {
                System.out.println("Record Save Success.....");
            }
        }
    }
}

```

```

        } else {
            System.out.println("Some problem is there.....");
        }
    } else {
        System.out.println("Database not connected");
    }
}
}
}

```

Example: WAP to verify user present in database or not using its username and password.

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class LoginVerificationApplication {
    public static void main(String[] args) throws SQLException {
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter username and password");
            String username=xyz.nextLine();
            String password=xyz.nextLine();
            Statement stmt=conn.createStatement();
            ResultSet rs=stmt.executeQuery("select *from register where
username='"+username+"' and password='"+password+"'");
            if(rs.next()) {
                System.out.println("Login Success.....");
            }
            else {
                System.out.println("Login Failed.....");
            }
        }
        else {
            System.out.println("Database is not connected");
        }
    }
}
}

```


ResultSet Methods : we have divided ResultSet methods in 4 categories

- **Navigational Methods**
- **Getter/Reader Methods**
- **Setter/Updater Methods**
- **Miscellaneous Methods** - close() and getMetaData()

1. ResultSet Navigational Methods

- **boolean absolute(int row) throws SQLException** **: ** This method moves ResultSet cursor to the specified row and returns true if the operation is successful.
- **void afterLast() throws SQLException** **: ** This method moves ResultSet cursor to the position after the last row.
- **void beforeFirst() throws SQLException** **: ** This method moves ResultSet cursor to the position before the first row.
- **boolean first() throws SQLException**: This method moves ResultSet cursor to the first row.
- **boolean last() throws SQLException**: This method moves ResultSet cursor to the last row.
- **boolean next() throws SQLException**: This method moves ResultSet cursor to the next row.
- **boolean previous() throws SQLException**: This method moves ResultSet cursor to the previous row.

Example:

```
package com.journaldev.examples;
```

```
import java.sql.*;
```

```
public class ResultSetDemo {
```

```
    public static void main(String[] args) {
```

```
        String query = "select empid, firstname, lastname, dob from tblemployee";
```

```
        Connection conn = null;
```

```
        Statement stmt = null;
```

```
        try {
```

```
            Class.forName("com.mysql.jdbc.Driver");
```

```
            conn = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/empdb",
```

```
"root", "root");
```

```
            stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
```

```
ResultSet.CONCUR_READ_ONLY);
```

```
            ResultSet rs = stmt.executeQuery(query);
```

```
            System.out.println("All the rows of table=>");
```

```
            while (rs.next()) {
```

```
                // Go to next row by calling next() method
```

```
                displayData(rs);
```

```
            }
```

```
            System.out.println("Now go directly to 2nd row=>");
```

```
            rs.absolute(2); // Go directly to 2nd row
```

```
            displayData(rs);
```

```
            System.out.println("Now go to Previous row=>");
```

```

        rs.previous();
        // Go to 1st row which is previous of 2nd row
        displayData(rs);
        rs.close();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {
            stmt.close();
            conn.close();
        } catch (Exception e) {
        }
    }
}

public static void displayData(ResultSet rs) throws SQLException {
    System.out.println("empId:" + rs.getInt(1));
    System.out.println("firstName:" + rs.getString(2));
    System.out.println("lastName:" + rs.getString(3));
    System.out.println("dob:" + rs.getDate(4));
    System.out.println("");
}
}

```

Output:

```

All the rows of table=>
empId:1
firstName:Mike
lastName:Davis
dob:1998-11-11

empId:2
firstName:Josh
lastName:Martin
dob:1988-10-22

empId:3
firstName:Ricky
lastName:Smith
dob:1999-05-11

Now go directly to 2nd row=>
empId:2
firstName:Josh
lastName:Martin
dob:1988-10-22

Now go to Previous row=>
empId:1
firstName:Mike
lastName:Davis
dob:1998-11-11

```

2. ResultSet Getter/Reader Methods

- **int getInt(int columnIndex) throws SQLException:**
This method returns value of specified columnIndex as **int**.

- **long** getLong(**int** columnIndex) **throws** SQLException: This method returns value of specified columnIndex as **long**
- String getString(**int** columnIndex) **throws** SQLException: This method returns value of specified columnIndex as **String**
- java.sql.Date getDate(**int** columnIndex) **throws** SQLException: This method returns value of specified columnIndex as **java.sql.Date**
- **int** getInt(String columnLabel) **throws** SQLException: This method returns value of specified column name as **int**.
- **long** getLong(String columnLabel) **throws** SQLException: This method returns value of specified column name as **long**.
- String getString(String columnLabel) **throws** SQLException: This method returns the value of the specified column name as String.
- java.sql.Date getDate(String columnLabel) **throws** SQLException: This method returns the value of the specified column name as **java.sql.Date**.
- ResultSet contains getter methods that return other primitive datatypes like boolean, float and double. It also has methods to obtain array and binary data from the database.

3. ResultSet Setter/Updater Methods

- **void** updateInt(**int** columnIndex, **int** x) **throws** SQLException: This method updates the value of specified column of current row with **int** value.
- **void** updateLong(**int** columnIndex, **long** x) **throws** SQLException: This method updates the value of the specified column of the current row with long value.
- void updateString(**int** columnIndex, String x) **throws** SQLException: This method updates the value of the specified column of the current row with a String value.
- **void** updateDate(**int** columnIndex, java.sql.Date x) **throws** SQLException: This method updates the value of specified column of current row with java.sql.Date value.
- void updateInt(String columnLabel, **int** x) **throws** SQLException: This method updates the value of the specified column label of the current row with int value.
- void updateLong(String columnLabel, **long** x) **throws** SQLException: This method updates the value of the specified column label of the current row with long value.
- void updateString(String columnLabel, String x) **throws** SQLException: This method updates the value of the specified column label of the current row with a String value.
- **void** updateDate(String columnLabel, java.sql.Date x) **throws** SQLException: This method updates the value of specified columnLabel of current row with java.sql.Date value.

Note: Setter/Updater Methods doesn't directly update database values. **Database values will be inserted/updated after calling the insertRow or updateRow method.**

Example:

```
package com.journaldev.examples;
import java.sql.*;
```

```
public class ResultSetUpdateDemo {
    public static void main(String[] args) {
        String query = "select empid, firstname, lastname, dob from tblemployee";
        Connection conn = null;
        Statement stmt = null;
        try {
```

```

        Class.forName("com.mysql.jdbc.Driver");
        conn = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/empdb",
"root", "root");

        stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);

        ResultSet rs = stmt.executeQuery(query);
        System.out.println("Now go directly to 2nd row for Update");
        if (rs.absolute(2)) {
            // Go directly to 2nd row
            System.out.println("Existing Name:" + rs.getString("firstName"));
            rs.updateString("firstname", "Tyson");
            rs.updateRow();
        }
        rs.beforeFirst(); // go to start
        System.out.println("All the rows of table=>");
        while (rs.next()) {
            // Go to next row by calling next() method
            displayData(rs);
        }
        rs.close();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {
            stmt.close();
            conn.close();
        } catch (Exception e) {
        }
    }
}

public static void displayData(ResultSet rs) throws SQLException {
    System.out.println("empId:" + rs.getInt(1));
    System.out.println("firstName:" + rs.getString(2));
    System.out.println("lastName:" + rs.getString(3));
    System.out.println("dob:" + rs.getDate(4));
    System.out.println("");
}
}

```

Output:

```

Now go directly to 2nd row for Update
Existing Name:Josh
All the rows of table=>
empId:1
firstName:Mike
lastName:Davis
dob:1998-11-11

empId:2
firstName:Tyson
lastName:Martin
dob:1988-10-22

empId:3
firstName:Ricky
lastName:Smith
dob:1999-05-11

```

4. ResultSet Miscellaneous Methods

- **void close() throws SQLException**:**** This method frees up resources associated with ResultSet Instance. It must be called otherwise it will result in resource leakage.
- **ResultSetMetaData getMetaData() throws SQLException:** This method returns ResultSetMetaData instance. It gives information about the type and property of columns of the query output.

PreparedStatement interface

Q. What is PreparedStatement interface?

PreparedStatement interface is used for performing the all DDL and DML operation on database table like as Statement interface.

Prepared Statement is used to execute same SQL statements repeatedly. The prepared statement is compiled only once even though it used "n" number of times.

Q. What is diff between Statement and PreparedStatement?

Statement	PreparedStatement
It is used when we want to execute query or SQL statement only once	It is recommended when we want to execute SQL Query more than one time.
You cannot pass parameter at run time and for run time parameter need to use SQL injection and it is very tedious task when we have large number of parameter as well as SQL injection is not secure way to pass parameter to database	Using PreparedStatement we can pass run time parameter to SQL statement using methods not need to perform SQL Injections.
It is recommended when we execute create , alter or drop statements	It is recommended for DML statements
Perform is very low	Performance is better
Used for execute static query or normal query	Used for execute dynamic query or run time parameter query

You cannot use Statement for execute binary data like as image , audio etc	You can use to pass binary data to database Like as storing image in database etc

Note: PreparedStatement is child of Statement interface

How to work with PreparedStatement interface in JDBC

a) Add java.sql package in application

```
import java.sql.*;
```

b) Create reference of PreparedStatement interface

If we want to create reference of PreparedStatement interface we have method of Connection name as `prepareStatement()`

Syntax: `PreparedStatement ref = connref.prepareStatement(String sqlStatement);`

Example: We have employee table with field id,name and salary and input we want to accept input from keyboard and store in database table using PreparedStatement interface.

short code(consider we have conn reference)

```
PreparedStatement stmt=conn.prepareStatement("insert into employee values(?,?,?)");
```

Note: if we think about above code we have ? in SQL statement and ? indicate the parameter to SQL Statement and if we want to pass parameter we have some setter method in PreparedStatement shown in below code.

Methods of PreparedStatement interface for pass run time parameter to SQL Statement

void setInt(int index, int data): this method can pass run time integer parameter to table column using its index and index start from 1 and in query first ? mark consider as first parameter from left to right

void setFloat(int index,float data): this method can pass float type of data to SQL Statement

void setString(int index,String data): this method can pass string type of data to SQL statement

void setDouble(int index, double data): this method can pass double type of data to SQL statement

void setLong(int index,long data): this method can pass long type of data to SQL Statement

void setShort(int index,short data)

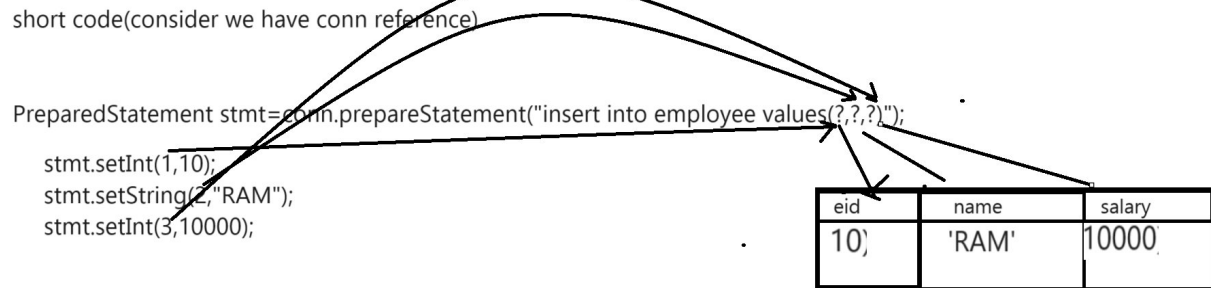
void setCharacterString(int index,Reader,int length): this method pass character stream data to column

void setBinaryStream(int index,InputStream,int length); this method can pass binary stream data to column

short code(consider we have conn reference)

```
PreparedStatement stmt=conn.prepareStatement("insert into employee values(?,?,?)");
```

```
stmt.setInt(1,10);  
stmt.setString(2,"RAM");  
stmt.setInt(3,10000);
```



eid	name	salary
10	'RAM'	10000

c) Use its executeUpdate() and executeQuery() method for execute DML statements

int executeUpdate(): this method help us to execute the all ddl and DML statement except select if statement executed successfully return 1 otherwise return 0

ResultSet executeQuery(): this method can execute only select query.

Example: input name id and salary from keyboard and store in employee table using JDBC with PreparedStatement

```
package org.techhub;  
import java.sql.*;  
import java.util.*;  
public class PstmtApplication {  
    public static void main(String[] args)throws SQLException {  
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();  
        DriverManager.registerDriver(d);  
        Connection  
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");  
        if(conn!=null) {  
            Scanner xyz = new Scanner(System.in);  
            System.out.println("Enter name id and salary of employee");  
            String name=xyz.nextLine();  
            int id=xyz.nextInt();  
            int salary=xyz.nextInt();  
            PreparedStatement stmt=conn.prepareStatement("insert into employee values(?,?,?)");  
            stmt.setInt(1, id);  
            stmt.setString(2, name);  
            stmt.setInt(3, salary);  
            int value=stmt.executeUpdate();  
            if(value>0) {  
                System.out.println("Employee Added Successfully.....");  
            }  
            else {  
                System.out.println("Some problem is there.....");  
            }  
        }  
    }  
}
```

```

    }
}
else {
    System.out.println("Database is not connected");
}
}
}

```

Example: WAP to input delete employee record by using its id using PreparedStatement interface

```

package org.techhub;
import java.util.*;
import java.sql.*;
public class DeleteEmployeeByIdUsingPstmtApplication {
    public static void main(String[] args) throws SQLException {
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            System.out.println("Connected");
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter employee id");
            int id=xyz.nextInt();
            PreparedStatement stmt=conn.prepareStatement("delete from employee where eid=?");
            stmt.setInt(1, id);
            int value=stmt.executeUpdate();
            if(value>0) {
                System.out.println("Record Deleted Successfully.....");
            }
            else {
                System.out.println("Some problem is there.....");
            }
        }
        else {
            System.out.println("Not Connected");
        }
    }
}

```

Example: WAP to change name and salary of employee using its id.

```

package org.techhub;
import java.sql.*;

```



```

import java.util.*;
public class UpdateUsingpstmtapp {
    public static void main(String[] args)throws SQLException {
        com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        DriverManager.registerDriver(d);
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter name id and salary of employee");
            String name=xyz.nextLine();
            int id=xyz.nextInt();
            int sal=xyz.nextInt();
            PreparedStatement stmt=conn.prepareStatement("update employee set name=?,salary=? where
eid=?");

            stmt.setString(1, name);
            stmt.setInt(2, sal);
            stmt.setInt(3, id);
            int value=stmt.executeUpdate();
            if(value>0) {
                System.out.println("Record updated");
            }
            else {
                System.out.println("Record Not Updated");
            }
        }
        else {
            System.out.println("Database is not connected");
        }
    }
}

```

How to select record from database table using PreparedStatement interface

Example: Fetch all employee data using PreparedStatement

```

package org.techhub;
import java.sql.*;
public class SelectRecordUsingPstmtApp {
    public static void main(String[] args)throws Exception {
        //com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        //DriverManager.registerDriver(d);
        Class.forName("com.mysql.cj.jdbc.Driver");
    }
}

```

```

        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            PreparedStatement stmt=conn.prepareStatement("select *from employee");
            ResultSet rs=stmt.executeQuery();
            while(rs.next()) {

System.out.println(rs.getInt("eid")+"\t"+rs.getString("name")+"\t"+rs.getInt("salary"));
            }
        }
        else {
            System.out.println("Not Connected");
        }
    }
}

```

How to use where clause in select using PreparedStatement interface

when we use where clause in PreparedStatement with select then after where clause parameter pass using PreparedStatement object using its method and before from you can column using ResultSet interface.

Example: WA Program fetch employee name using its id

Your query look like as

select name from employee where eid=?

so as per above query we need to pass eid using PreparedStatement interface and we get name using ResultSet because we use eid after where and we use name before from

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class SelectRecordUsingPstmtApp {
    public static void main(String[] args)throws Exception {
        //com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        //DriverManager.registerDriver(d);
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");

```

```

        if(conn!=null) {
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter employee id");
            int id=xyz.nextInt();
PreparedStatement stmt=conn.prepareStatement("select name from employee where eid=?");
            stmt.setInt(1, id);//where parameter
            ResultSet rs=stmt.executeQuery();
            while(rs.next()) {
                String name=rs.getString("name");//before from parameter
                System.out.println(name);
            }
        }
        else {
            System.out.println("Not Connected");
        }
    }
}

```

Example: WAP fetch user detail name, email and contact from register table using its username and password

```

package org.techhub;
import java.util.*;
import java.sql.*;
public class TestSelectWithWhere {
    public static void main(String[] args) throws Exception{
        //com.mysql.cj.jdbc.Driver d = new com.mysql.cj.jdbc.Driver();
        //DriverManager.registerDriver(d);
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Scanner xyz = new Scanner(System.in);
            System.out.println("Enter username and password");
            String username=xyz.nextLine();
            String password=xyz.nextLine();
            PreparedStatement stmt=conn.prepareStatement("select name,email,contact from
register where username=? and password=?");
            stmt.setString(1, username);
            stmt.setString(2, password);
            ResultSet rs=stmt.executeQuery();

```

```

        if(rs.next()) {

            System.out.println(rs.getString("name")+"\t"+rs.getString("email")+"\t"+rs.getString("contact"));

        }
        else {

            System.out.println("Data not found");

        }

    }
    else {

        System.out.println("Database is not connected");

    }

}
}

```

CallableStatement interface

CallableStatement interface is used for call database procedure using JDBC.

Steps to work with CallableStatement interface

1) Add java.sql package in application

```
import java.sql.*;
```

2) Create Reference of CallableStatement interface and call procedure

If we want to create reference of CallableStatement interface we have prepareCall() method of Connection and this method return reference of CallableStatement.

Syntax: CallableStatement ref = connref.prepareCall("{call procedurename(?,?,?.....?)}");

Example: We want to create procedure name as saveEmployee and call it using JDBC Program

```
mysql> delimiter //
mysql> create procedure saveEmployee(IN id int(5),IN name varchar(200),sal int(5))
-> begin
-> insert into employee values(id,name,sal);
-> end
-> //
Query OK, 0 rows affected, 2 warnings (0.06 sec)
```

```
package org.techhub;
```

```
import java.sql.*;
```

```
import java.util.*;
```

```

public class CallProApp {
    public static void main(String[] args)throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Scanner xyz =new Scanner(System.in);
            System.out.println("Enter name id and salary of employee");
            String name=xyz.nextLine();
            int id=xyz.nextInt();
            int salary=xyz.nextInt();
            CallableStatement cstmt=conn.prepareCall("{call saveEmployee(?,?,?)}");
        }
        else {
            System.out.println("Database is not connected");
        }
    }
}

```

If we think about above code we have ? in procedure and it is indicate parameter to procedure and if we want to pass parameter to procedure we have setXXX() method of PreparedStatement
All methods of PreparedStatement present in CallableStatement because CallableStatement is child of PreparedStatement

3) Pass parameter to procedure

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class CallProApp {
    public static void main(String[] args)throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Scanner xyz =new Scanner(System.in);
            System.out.println("Enter name id and salary of employee");
            String name=xyz.nextLine();
            int id=xyz.nextInt();
            int salary=xyz.nextInt();
            CallableStatement cstmt=conn.prepareCall("{call saveEmployee(?,?,?)}");
            //set parameter to procedure

```

```

        cstmt.setInt(1, id);
        cstmt.setString(2, name);
        cstmt.setInt(3, salary);

    }
    else {
        System.out.println("Database is not connected");
    }
}
}

```

4) execute procedure : if we want to execute the procedure we have execute() method of CallableStatement and this method return false if procedure executed successfully otherwise return true.

boolean execute():

```

package org.techhub;
import java.sql.*;
import java.util.*;
public class CallProApp {
    public static void main(String[] args)throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbcmayjunejuly","root","root");
        if(conn!=null) {
            Scanner xyz =new Scanner(System.in);
            System.out.println("Enter name id and salary of employee");
            String name=xyz.nextLine();
            int id=xyz.nextInt();
            int salary=xyz.nextInt();
            CallableStatement cstmt=conn.prepareCall("{call saveEmployee(?,?,?)}");
            //set parametre to procedure
            cstmt.setInt(1, id);
            cstmt.setString(2, name);
            cstmt.setInt(3, salary);
            boolean b=cstmt.execute();
            if(!b) {
                System.out.println("Procedure executed successfully.....");
            }
        }
    }
}

```

```
        else {  
            System.out.println("Some problem is there.....");  
        }  
    }  
    else {  
        System.out.println("Database is not connected");  
    }  
}  
}
```

6. Close the Database Connection

Student Examination

Objective: Student should give exam on system and system should generate result of exam to students.

Basic Requirement

1. Add Subjects in Database
2. Add Question under subjects
3. Create Exam Time Table
4. Add Question under exam
5. Add New Student in Database and provide login
6. Student should login to system
7. Attempt exam by students
8. Show the Result of Student

Database Design

StudentExamRelation

stid	schid	obtainper	status
1	1	0	0

Table Structure

Subject Table

Column Name	Data Type	Size	Constraints
Sid	Int	5	Primary key auto_increment
Subjectname	Varchar	200	Not null unique

Question Table

Column Name	Data Type	Size	Constraints
Qid	Int	5	Primary key auto_increment
Question	Varchar	4000	Not null unique
Op1	Varchar	200	Not null
Op2	Varchar	200	Not null
Op3	Varchar	200	Not null
Op4	Varchar	200	Not null
Answer	Int	5	Not null

SubjectQuestionJoin

Column Name	Data Type	Size	Constraints
Qid	Int	5	Foreign key on delete cascade
Sid	Int	5	Foreign key on delete cascade on update cascade

Exam Table

Column Name	Data Type	Size	Constraints
Examid	Int	5	Primary key auto_increment
Examname	Varchar	200	Not null unique
Totalmarks	Int	5	Not null
Passingmarks	Int	5	Not null

Schedule Table

Column Name	Data Type	Size	Constraints
Schid	Int	5	Primary key auto_increment
Examid	Int	5	Foreign key on delete cascade on update cascade
Date	Date		Not null
Starttime	Time		Not null
Endtime	Time		Not null
Sid	Int	5	Foreign key on delete cascade on update cascade

Student Table

Column Name	Data Type	Size	Constraints
Stid	Int	5	Primary key
Name	Varchar	200	Not null
Email	Varchar	200	Not null unique
Contact	Varchar	200	Not null unique
Username	Varchar	200	Not null unique
Password	Varchar	200	Not null unique

StudentSubjectJoinTable

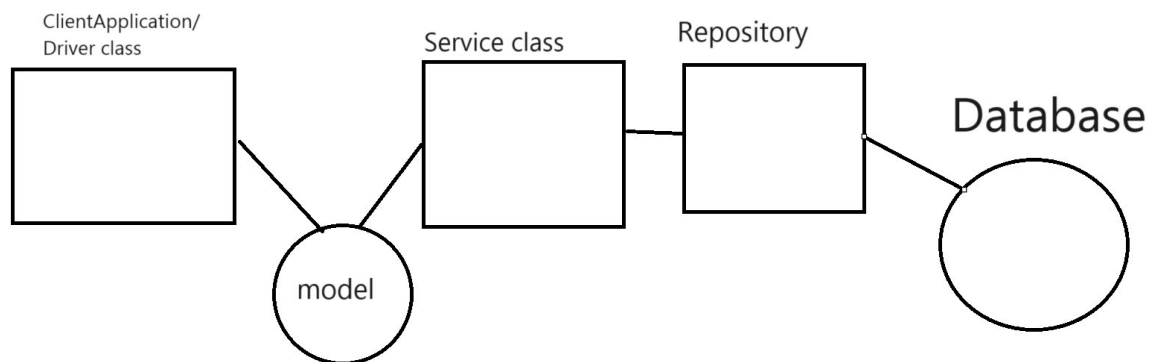
Column Name	Data Type	Size	Constraints
Stid	Int	5	Foreign key on delete cascade on update cascade
Sid	Int	5	Foreign key on delete cascade on update cascade

StudentExamRelation

Column Name	Data Type	Size	Constraints
Stid	Int	5	Foreign key on delete cascade on update cascade
Schid	Int	5	Foreign key on delete

			cascade on update cascade
Obtainper	Int	5,5	Not null default 0.0
Status	Int	5	Default 0

Project Code Architecture



Client Application: ClientApplication means class with main method from user provide the input and get results.

Model: Model class means class with setter and getter method which is used for store data in object and pass to service layer and from service layer pass to repository layer.

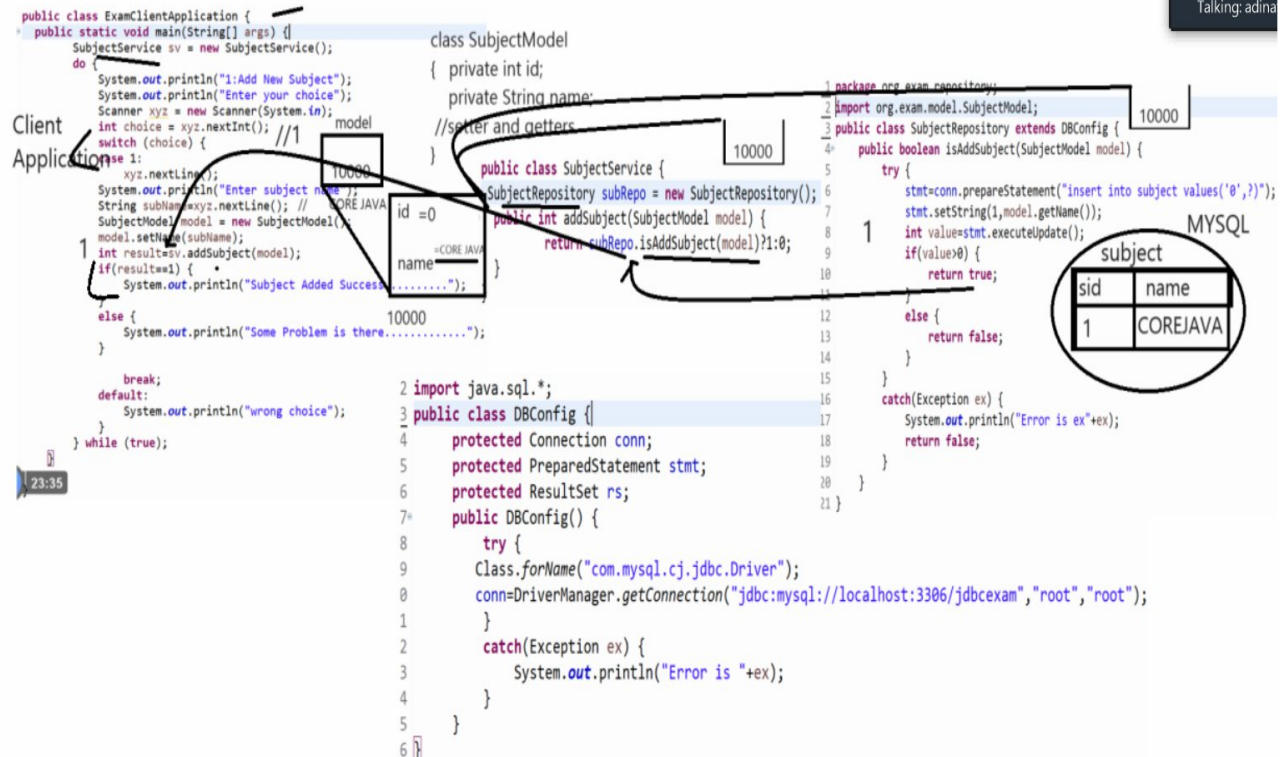
Service layer or service class: Service layer or service class is intermediate class which we can use for writing business logics.

Q. What are business logics?

Business logic may be anything which is dependent on requirement of application

Repository/DAO: Repository is class which is used for write the database query or database logics.

Note: repository class object should be create in service class and repository methods should call from service class and service class object should be create in client application and means service method should call from client and model object also create in client application and pass as parameter to service method from client and from service to repository methods



How to load the database credential using .properties file

Q. What is .properties file?

.properties file is facility in java which is used for store data or store hard coded data required for java application in the form of name and value pair.

Q. Why use .properties file?

Some time if we have hard coded data in application and if your data may be change in future and set your data in java code then drawback is when we want to change data we need to modify java code and again recompile java program and it is not possible every time when your application is running on server so we want to change data hard coded without touching java code as well as without recompile java code then we can set data in .properties file

Note: .properties file use by java developer to write the configuration of java application

Example: database configuration etc

How to store data in .properties file

If we want to store data in .properties file we can store in the form of name and value pair.

Syntax: name=value

Example:

```
driverClassName=com.mysql.cj.jdbc.Driver
userName=root
password=root
url=jdbc:mysql://localhost:3306/jdbcexam
```

Q. How to load the .properties file in Java Application & get its data?

If we want to load the .properties file Java Application we have Properties class from java.util package

Source code

```
package org.techhub;
import java.io.FileInputStream;
import java.util.Properties;
import java.io.IOException;
import java.nio.file.FileSystems;
import java.nio.file.Path;
public class PropLoadingApplication {
    public static void main(String[] args) throws IOException {
        Path currentDirectoryPath = FileSystems.getDefault().getPath("");
        String currentDirectoryName = currentDirectoryPath.toAbsolutePath().toString();
        String completePath=currentDirectoryName+"\\src\\resources\\db.properties";
        System.out.println ("Current Directory = \"\" + completePath + "\"");
        Properties p = new Properties();
        //InputStream
        FileInputStream finf = new FileInputStream(completePath);
        p.load(finf);
        String driverClassName=p.getProperty("driver");
        String urlName=p.getProperty("url");
        String userName=p.getProperty("user");
        String passWord=p.getProperty("pass");
        System.out.println(driverClassName);
        System.out.println(userName);
        System.out.println(passWord);
        System.out.println(urlName);
    }
}
```

How to Store CSV file in Database using JDBC?

Q. what is CSV?

CSV stands for comm separated file means it is simple notepad file and we store data in it by separating commans and if we open this file in excel then every comm consider as column in excel and new line consider as row in excel.

Q. Why use CSV file?

1. Upload bulk data in database at time
2. Comma separation consider as column database table
3. It is light weighted format
4. It can open in notepad, wordpad as well as in excel also
5. Not need to use external API for separating data just you can manage data by using string handling

Q. How Create CSV File and store data in it?

Open notepad and save file using .csv extension and store data in it and separate data by comma



```
employee.csv
File Edit View
ram,10000
shyam,20000
ganesh,30000
dinesh,40000
rajesh,50000
mangesh,60000
```

Q. How load CSV file in database using JDBC?

1. Read file in java application by using Reader classes.

Example: You can use BufferedReader class

RowSet	ResultSet
RowSet is present in the javax.sql package	ResultSet is present in the java.sql package
A Row Set can be connected, disconnected from the database.	A ResultSet always maintains the connection with the database.
RowSet is scrollable providing more flexibility	ResultSet by default is always forward only
A Row Set object can be serialized.	It cannot be serialized.
You can pass a Row Set object over the network.	ResultSet object cannot be passed other over the network.
Result Set Object is a JavaBean object. RowSet using the RowSetProvider.newFactory().createJdbc cRowSet() method.	Result Set object is not a JavaBean object result set using the executeQuery() method
ResultSetBy default, RowSet object is scrollable and updatable.	By default, the ResultSet object is not scrollable or, updatable.
Faster than ResultSet	ResultSet slower than Rowset.

RowSet :

RowSet is an interface in java that is present in the java.sql package. Geek do note not to confuse RowSet with ResultSet.

Note: RowSet is present in package javax.sql while ResultSet is present in package java.sql.

The instance of RowSet is the java bean component because it has properties and a java bean notification mechanism. It is introduced in JDK5. A JDBC RowSet provides a way to store the data in tabular form. It makes the data more flexible and easier than a ResultSet. The connection between the RowSet object and the data source is maintained throughout its life cycle.

RowSets are classified into five categories based on how they are implemented which are listed namely as below:

- JdbcRowSet
- CachedRowSet
- WebRowSet
- FilteredRowSet

- JoinRowSet

The advantage of RowSet is as follows:

1. It is easy and flexible to use.
2. It is by default scrollable and can be updated by default whereas ResultSet by default is only forwardable and read-only operation is valid there only.

The JDBC RowSet interface is a RowSet extension. It's a wrapper for the ResultSet object that adds some extra features.

Syntax: Declaration of Jdbc RowSet interface

```
public interface JdbcRowSet extends RowSet, Joinable
```

In order to connect RowSet with the database, the RowSet interface provides methods for configuring Java bean properties which are depicted below:

```
void setURL(String url):
```

```
void setUsername(String username):
```

```
void setPassword(String password):
```

Example:

```
// Java Program to Illustrate RowSet in JDBC
```

```
// Importing database
```

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
import java.sql.ResultSet;
```

```
import java.sql.Statement;
```

```
import javax.sql.RowSetEvent;
```

```
import javax.sql.RowSetListener;
```

```
import javax.sql.rowset.JdbcRowSet;
```

```
import javax.sql.rowset.RowSetProvider;
```

```
// Main class
```

```
class RowSetDemo {
```

```
    // Main driver method
```

```
    public static void main(String args[])
```

```
    {
```

```
        // Try block to check for exceptions
```

```
        try {
```

```
            // Loading and registering drivers
```

```
            Class.forName("oracle.jdbc.driver.OracleDriver");
```

```
            // Creating a RowSet
```



```

JdbcRowSetrowSet = RowSetProvider.newFactory().createJdbcRowSet();

// Setting URL, username, password
rowSet.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
rowSet.setUsername("root");
rowSet.setPassword("pass");

// Creating a query
rowSet.setCommand("select * from Student");

// Executing the query
rowSet.execute();

// Processing the results
while (rowSet.next()) {
    // Print and display commands
    System.out.println("RollNo: "+ rowSet.getInt(1));
    System.out.println("Name: "+ rowSet.getString(2));
    System.out.println("Marks: "+ rowSet.getString(3));
}
}
// Catch block to handle the exceptions
catch (Exception e) {
    // Print and display the exception along with
    // line number using printStackTrace() method
    e.printStackTrace();
}
}
}

```

Output:

```

RollNo: 1
Name: jack
Marks: 92
RollNo: 2
Name: jenny
Marks: 90
RollNo: 3
Name: mark
Marks: 80
RollNo: 4
Name: joe
Marks: 82

```

Why would you use `setAutoCommit(false)` in JDBC?

If you want to turn Off the Auto Commit then set `connection.setAutoCommit(false)`.

Example:

```
Class.forName(drivers);
Connection dbConnnection=DriverManager.getConnection(connectionURL,username,password);
dbConnection.setAutoCommit(false); //Disabling the Autocommit
Statement selectStatement = dbConnection.createStatement("Select Query");
ResultSet rs = selectStatement.execute();
while(rs.next()){
    Statement updateStatement = dbConnection.createStatement("update Query");
    //Apply some changes to update record
    statement.execute();
    dbConnection.commit(); //Mandatory to execute to persist changes into database
}
```

Disabling Auto-Commit Mode

When a connection is created, it is in auto-commit mode. This means that each individual SQL statement is treated as a transaction and is automatically committed right after it is executed. (To be more precise, the default is for a SQL statement to be committed when it is completed, not when it is executed. A statement is completed when all of its result sets and update counts have been retrieved. In almost all cases, however, a statement is completed, and therefore committed, right after it is executed.)

The way to allow two or more statements to be grouped into a transaction is to disable the auto-commit mode. This is demonstrated in the following code, where `con` is an active connection:

```
con.setAutoCommit(false);
```